

ÁRVORES

Estrutura de Dados

Prof. Dr. Clayton Kossoski



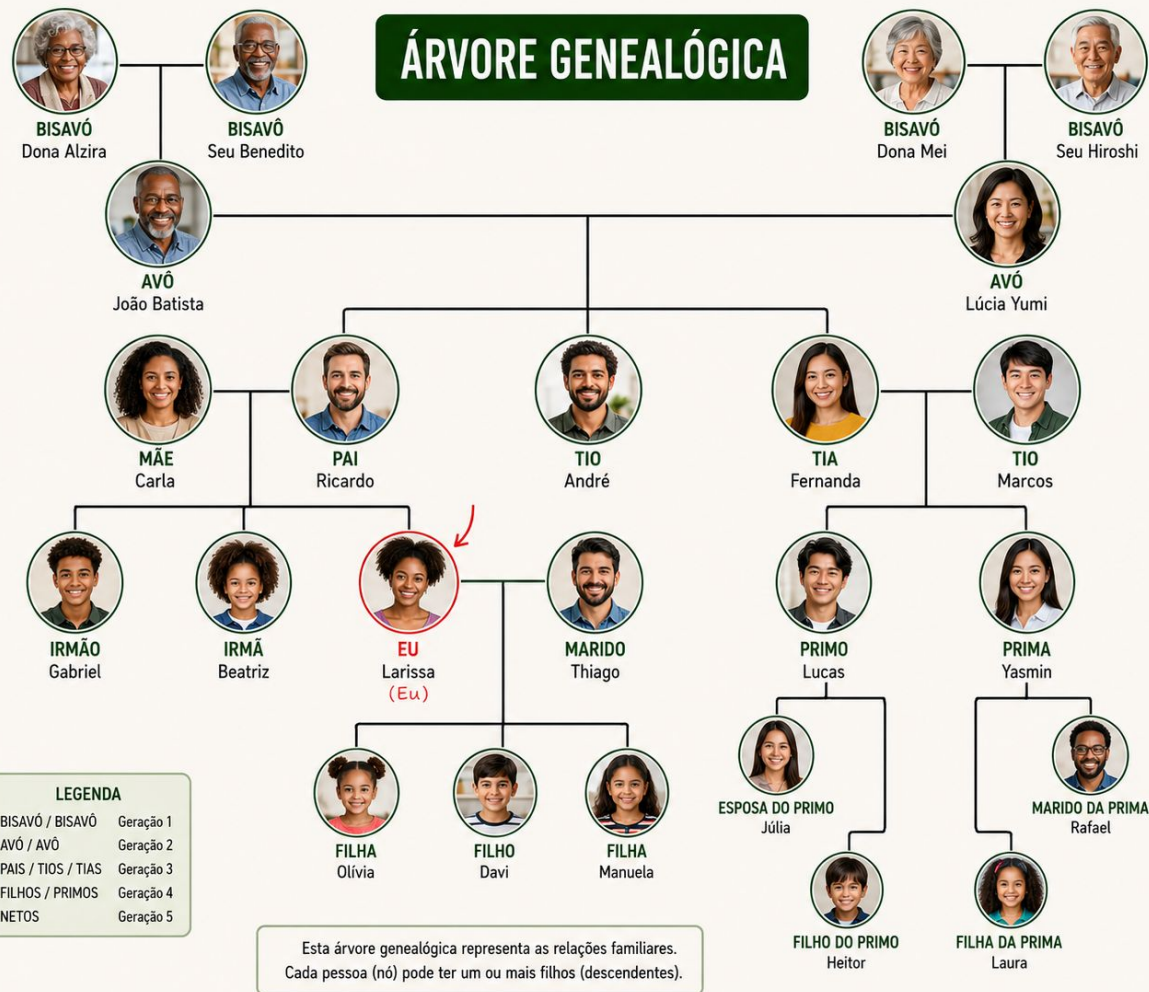
Definição

Diversas aplicações necessitam representar conjuntos de objetos e suas relações hierárquicas.

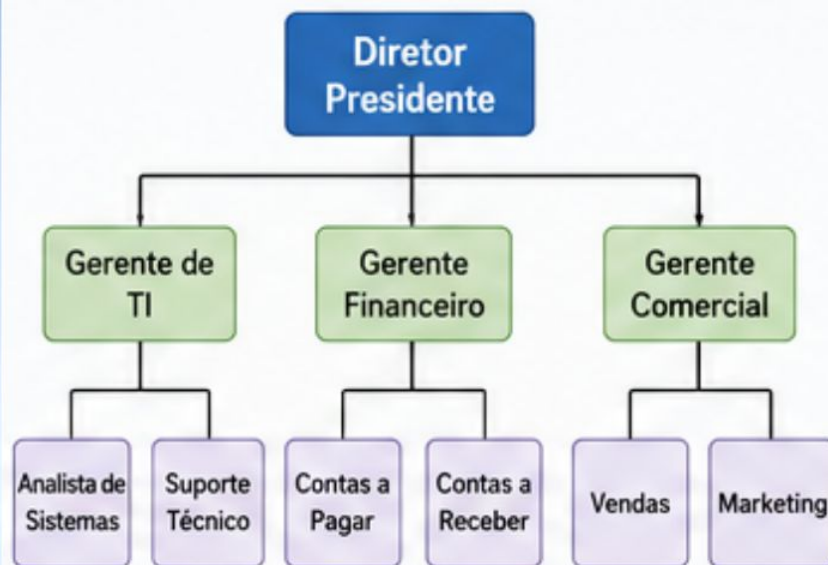
Uma árvore é uma abstração matemática utilizada para modelar estruturas hierárquicas de forma eficiente.

Diferentemente das estruturas lineares, as árvores são estruturas de dados não lineares, compostas por nós interligados que representam relações de ancestralidade e descendência entre os elementos.

Exemplos de estruturas hierárquicas

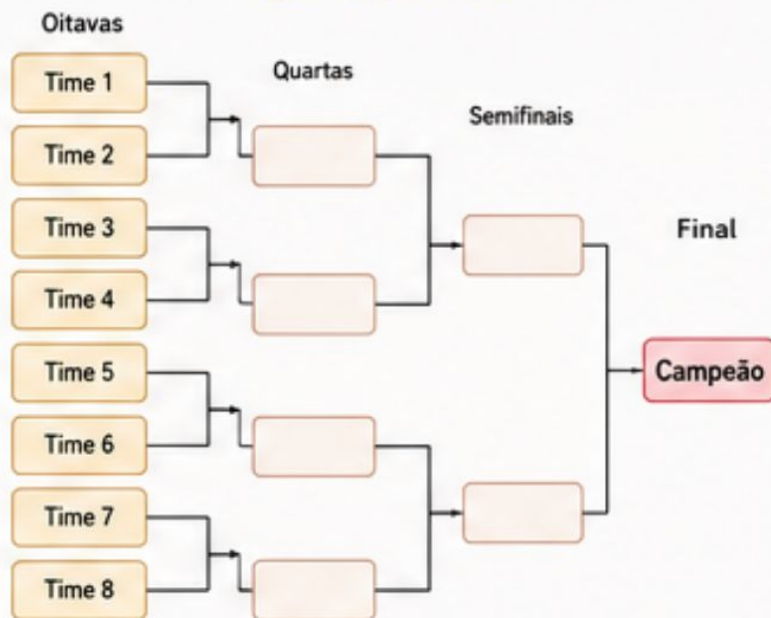


2. DIAGRAMA HIERÁRQUICO DE UMA ORGANIZAÇÃO



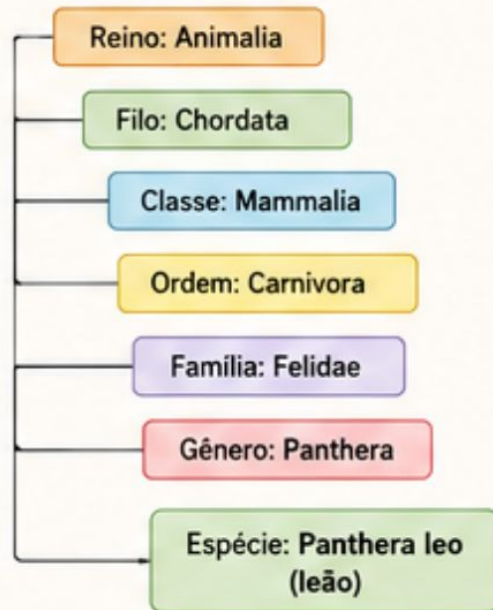
Representa a estrutura administrativa da empresa, com níveis de autoridade e subordinação.

3. CAMPEONATOS ESPORTIVOS



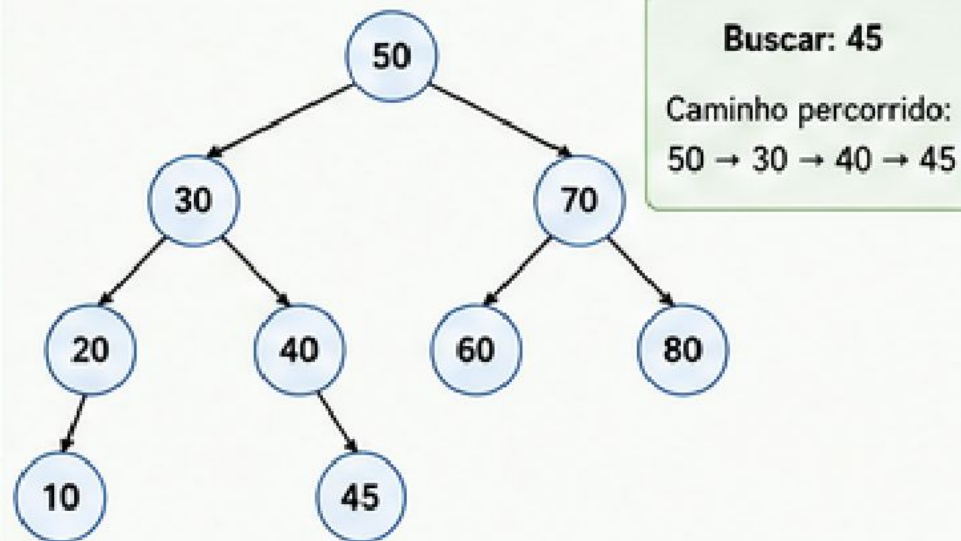
Em torneios eliminatórios, cada partida gera um vencedor que avança para a próxima fase.

4. TAXONOMIA (CLASSIFICAÇÃO DE SERES VIVOS)



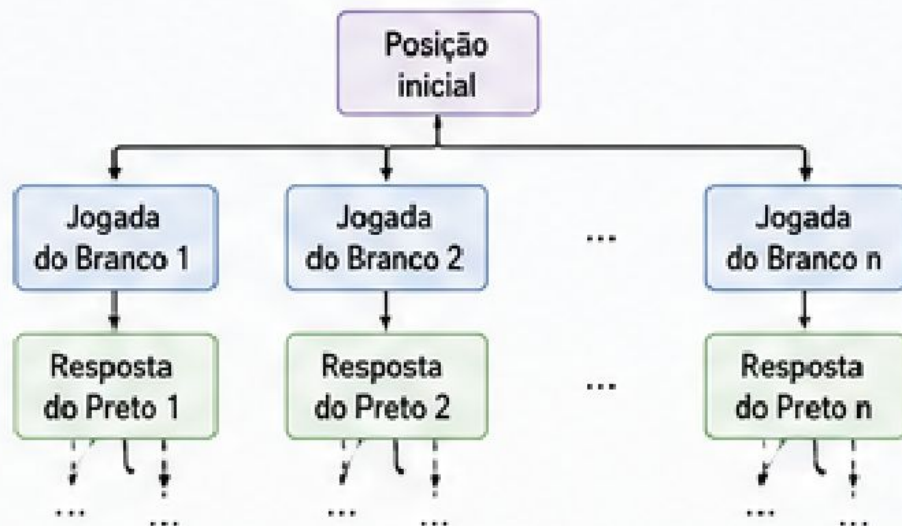
Organiza os seres vivos em categorias do geral para o específico.

5. BUSCA DE DADOS ARMAZENADOS NO COMPUTADOR



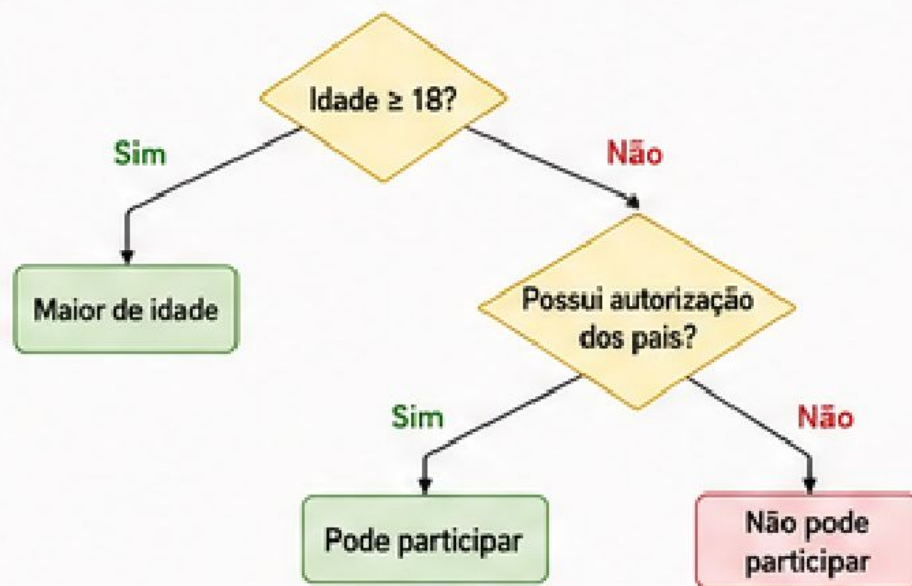
Árvores (como a Árvore Binária de Busca) permitem localizar dados de forma eficiente.

6. REPRESENTAÇÃO DE ESPAÇO DE SOLUÇÕES (EX.: JOGO DE XADREZ)



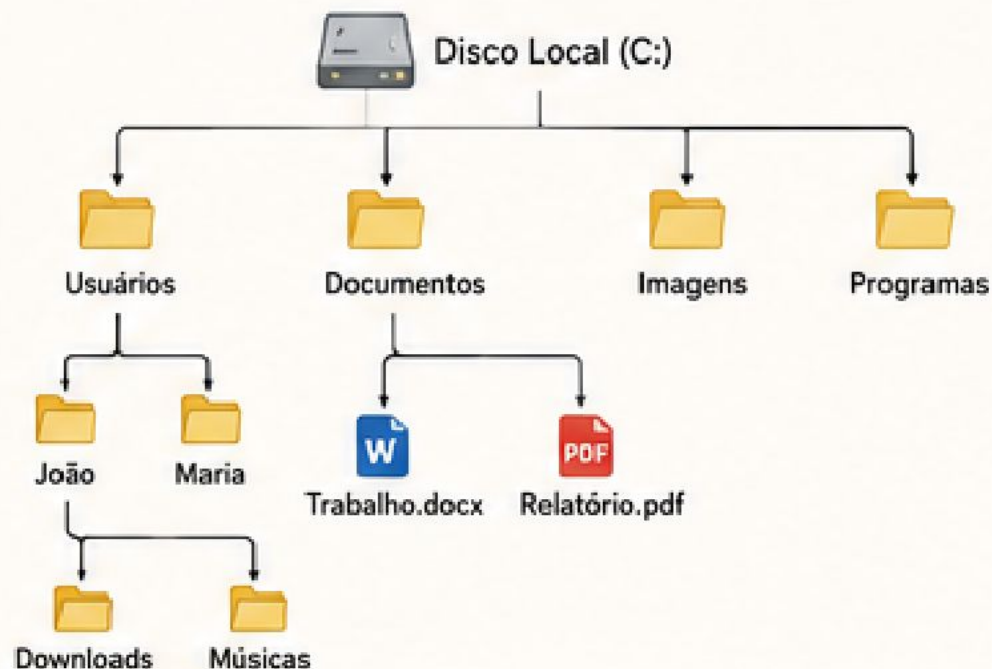
Cada nó representa um estado do jogo e as arestas, as jogadas possíveis.

7. MODELAGEM DE ALGORITMOS (ÁRVORES DE DECISÃO)



Utilizada para representar decisões e fluxos lógicos em algoritmos e sistemas.

8. SISTEMAS DE ARQUIVOS



Pastas e arquivos são organizados hierarquicamente, facilitando o armazenamento e a busca.

Conceitos de grafo

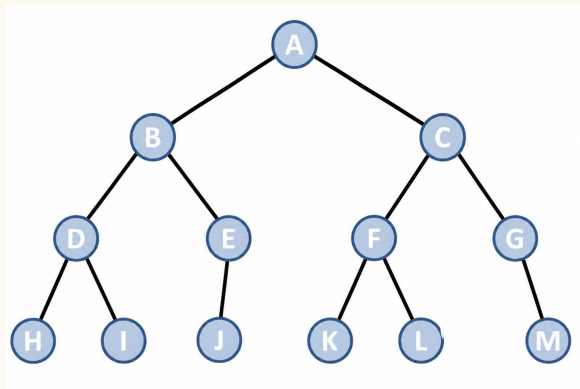
Árvore como um tipo especial de grafo

Uma árvore é um tipo especial de grafo.

É definida por um conjunto de nós (ou vértices) e arestas.

Existe exatamente um caminho entre quaisquer dois vértices da árvore.

Caracteriza-se por ser um grafo não direcionado, conexo e acíclico, ou seja, todos os vértices estão conectados e não há ciclos.



A HISTÓRIA DOS GRAFOS

De um problema de pontes a uma teoria fundamental da computação moderna



Grafos são estruturas formadas por vértices (nós) conectados por arestas (ligações). Eles surgiram para resolver problemas reais e, ao longo dos séculos, se tornaram essenciais na matemática, ciência da computação, engenharia e em muitas aplicações do dia a dia.

1736 – O INÍCIO

Leonhard Euler
O problema das pontes de Königsberg



Euler modelou o problema como um grafo e foi o primeiro a estudar propriedades abstratas dessas estruturas.

Nasce, assim, a Teoria dos Grafos.

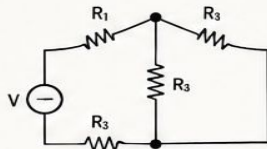
1736

1847 – A BASE MATEMÁTICA



Gustav Kirchhoff

Usou grafos para resolver problemas de circuitos elétricos. Introduziu a ideia de árvores geradoras.



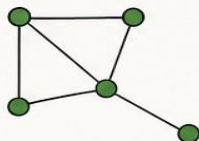
1847

1852 – SISTEMATIZAÇÃO



Francis Guthrie

Publicou o primeiro livro sobre teoria dos grafos. Foi o primeiro a usar o termo "graph".



1852

1878 – APLICAÇÕES PRÁTICAS



Arthur Cayley

Estudou contagem de árvores e estruturas químicas usando grafos.

Seu trabalho influenciou química, física e matemática.



1878

SÉCULO XX – EXPANSÃO



- ✓ Aplicações em logística, telecomunicações, transporte e redes elétricas.
- ✓ Desenvolvimento de algoritmos (Dijkstra, 1959), fluxo máximo, coloração de grafos, entre outros.
- ✓ Grafo se torna ferramenta central da ciência da computação.

1900+

SÉCULO XXI – PRESENTE



- ✓ Redes sociais, mapas digitais, recomendações, inteligência artificial e biologia de redes.
- ✓ Grafos estão presentes em diversos sistemas que usamos todos os dias.

2020+



DE UMA IDEIA SIMPLES
A INFINITAS POSSIBILIDADES

O que começou com um problema de pontes se transformou em uma das teorias mais poderosas e versáteis da matemática e da computação.



Rotas e logística



Redes sociais



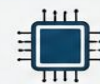
Bioinformática



Inteligência Artificial



Cibersegurança



Circuitos e sistemas



EM RESUMO

A teoria dos grafos evoluiu por séculos, guiada por problemas reais e pela criatividade de grandes mentes. Hoje, conecta pessoas, dados, máquinas e ideias em um mundo cada vez mais interligado.



O PROBLEMA DAS PONTES DE KÖNIGSBERG

O problema que deu origem à Teoria dos Grafos

1. A CIDADE E SUAS PONTES

Königsberg era atravessada pelo rio Pregel, que dividia a cidade em quatro áreas de terra: A, B, C e D.

Sete pontes conectavam essas áreas.



- | | |
|---------------------|--------------------|
| 1 Ponte do Ferreiro | 4 Ponte de Madeira |
| 2 Ponte Longa | 5 Ponte da Carne |
| 3 Ponte Verde | 6 Ponte Köttel |
| | 7 Ponte Alta |



2. O DESAFIO



Os moradores queriam saber se era possível começar em qualquer ponto, atravessar cada uma das 7 pontes exatamente uma vez e voltar ao ponto inicial.

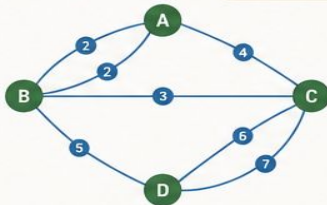
Será que existe um caminho assim?

3. REPRESENTANDO COMO UM GRAFO

Em 1736, Leonhard Euler teve uma ideia genial: representar o problema como um grafo.

Como funciona a representação?

- Cada área de terra vira um vértice (nó).
- Cada ponte vira uma aresta (ligação) entre os vértices correspondentes.



Grau dos vértices

O grau de um vértice é o número de pontes que saem dele.

A: 3 (1, 2, 4)
B: 3 (1, 2, 5)
C: 3 (3, 4, 6, 7)
D: 3 (5, 6, 7)

Todos os vértices têm grau ímpar.

4. A IDEIA DE EULER

Euler provou que um passeio que atravessa cada aresta (exatamente uma vez) só é possível se o grafo tiver:



0 vértices de grau ímpar → existe um circuito
(e o passeio começa e termina no mesmo vértice).



2 vértices de grau ímpar → existe um caminho
(e o passeio começa em um vértice ímpar e termina no outro).



Mais de 2 vértices de grau ímpar
→ não existe passeio possível nessas condições.

5. CONCLUSÃO PARA KÖNIGSBERG



Como o grafo de Königsberg tem 4 vértices de grau ímpar (A, B, C e D), não é possível atravessar cada ponte exatamente uma vez e retornar ao ponto de partida.

O passeio desejado é impossível!!



Leonhard Euler
(1707–1783)

6. O LEGADO



Esse problema simples foi o ponto de partida da Teoria dos Grafos, uma área da matemática que hoje é fundamental na ciência da computação, redes, engenharia, logística, biologia e muitas outras áreas.

APLICAÇÕES MODERNAS DA TEORIA DOS GRAFOS



Redes sociais



Rotas e logística



Circuitos elétricos e eletrônicos



Internet e redes de computadores



Bioinformática e biologia de redes

EM RESUMO

- ✓ Sete pontes, quatro terras e uma pergunta.
- ✓ Euler transformou o problema em um grafo.
- ✓ A análise dos graus mostrou que não há solução.
- ✓ Nascia ali uma nova forma de pensar e resolver problemas: a Teoria dos Grafos.



De um problema curioso do século XVIII surgiu uma das teorias mais poderosas e úteis da matemática moderna.
Tudo começou nas pontes de Königsberg.

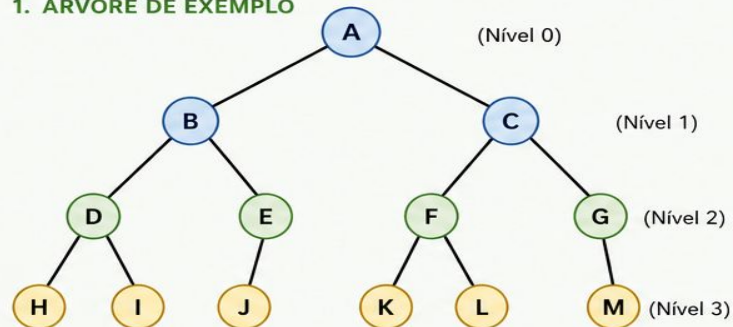
LEGENDA

● Vértice (área de terra) — Aresta (ponte)

Conceitos de árvores

ÁRVORE – EXEMPLO E CONCEITOS

1. ÁRVORE DE EXEMPLO



2. RAZI (ROOT)

É o nó que está no topo da árvore



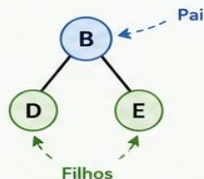
3. NÓ PAI E NÓ FILHO

Pai: nó que possui um ou mais filhos

Filho: nó que possui um pai

Exemplos:

- A é pai de B e C.
- B é pai de D e E.
- C é pai de F e G.
- D é pai de H e I.
- E é pai de J.
- F é pai de K e L.
- G é pai de M.

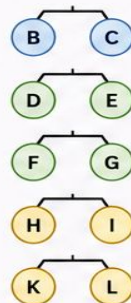


4. NÓS IRMÃOS

Nós que possuem o mesmo pai.

Exemplos:

- B e C
- D e E
- F e G
- H e I
- K e L



5. NÓ FOLHA (TERMINAL)

Nós que não possuem filhos.

Nós folha:



6. NÓS INTERNOS

Nós que possuem pelo menos um filho.

Nós internos:



7. GRAU DE UM NÓ

Quantidade de filhos que um nó possui.

Nó	A	B	C	D	E	F	G	H	I	J	K	L	M
Grau	2	2	2	2	1	2	1	0	0	0	0	0	0

8. NÍVEL DE UM NÓ

Distância da raiz até o nó.

Nível	0	1	2	3
Nós	A	B, C	D, E, F, G	H, I, J, K, L, M

9. ALTURA DA ÁRVORE

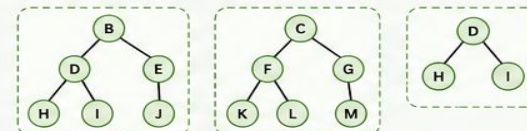
Maior nível existente na árvore.

Altura = 3

10. SUBÁRVORE

Subárvore é formada por um nó e todos os seus descendentes.

Exemplos de subárvores:



11. CAMINHO ENTRE DOIS NÓS

Sequência de nós conectados por arestas.

Exemplos:

- Caminho de A até M: A → C → G → M
- Caminho de H até L: H → D → B → A → C → F → L
- Caminho de J até K: J → E → B → A → C → F → K

12. ÁRVORE BINÁRIA?

Uma árvore é binária quando cada nó possui, no máximo, 2 filhos.

Esta árvore é binária.

13. QUANTIDADE DE NÓS E ARESTAS

Em uma árvore com n nós, sempre existem $(n - 1)$ arestas.

Quantidade de nós: 13 (A a M)

Quantidade de arestas: 12 ($13 - 1 = 12$)

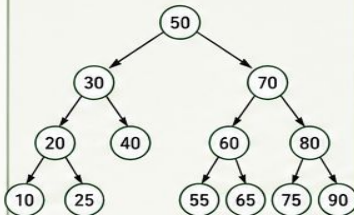
Tipos de árvores

TIPOS DE ÁRVORES

- Na computação, assim como na natureza, existem vários tipos diferentes de árvores.
- Cada uma delas foi desenvolvida pensando em diferentes tipos de aplicações

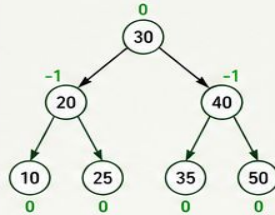
- árvore binária de busca
- árvore AVL
- árvore Rubro-Negra
- árvore B, B+ e B*
- árvore 2-3
- árvore 2-3-4
- quadtree
- octree

1. ÁRVORE BINÁRIA DE BUSCA



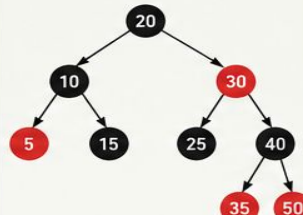
Cada nó à esquerda possui valor menor que o nó pai. Cada nó à direita possui valor maior que o nó pai.

2. ÁRVORE AVL



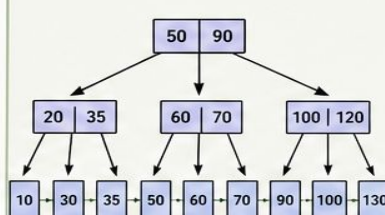
Árvore binária de busca balanceada. A altura das subárvores esquerda e direita de qualquer nó difere no máximo em 1.

3. ÁRVORE RUBRO-NEGRA



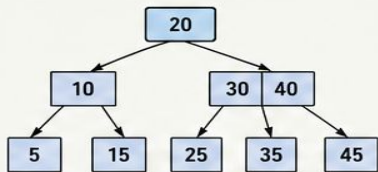
Árvore balanceada onde cada nó é vermelho ou preto, obedecendo regras que garantem o balanceamento.

4. ÁRVORES B, B+ E B*



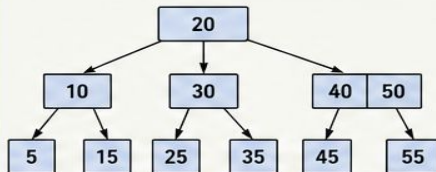
Árvores multi-vias usadas em sistemas de arquivos e bancos de dados para reduzir acessos ao disco.

5. ÁRVORE 2-3



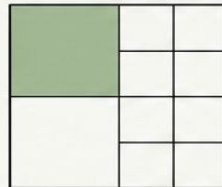
Cada nó pode ter 1 ou 2 chaves (e 2 ou 3 filhos). Mantém os dados ordenados e a árvore balanceada.

6. ÁRVORE 2-3-4



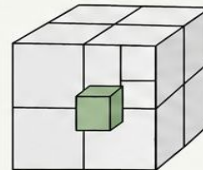
Cada nó pode ter 1, 2 ou 3 chaves (e 2, 3 ou 4 filhos). É uma generalização da árvore 2-3.

7. QUADTREE



Divide recursivamente um espaço 2D em 4 regiões (quadrantes). Útil em gráficos, imagens e jogos.

8. OCTREE



Divide recursivamente um espaço 3D em 8 regiões (octantes). Usado em gráficos 3D e simulações.



RESUMO

Cada tipo de árvore foi projetado para resolver problemas específicos com eficiência, equilibrando velocidade de acesso, uso de memória e facilidade de atualização.



Bancos de Dados



Busca e Indexação



Gráficos e Imagens



Jogos e Simulações



Modelagem 3D



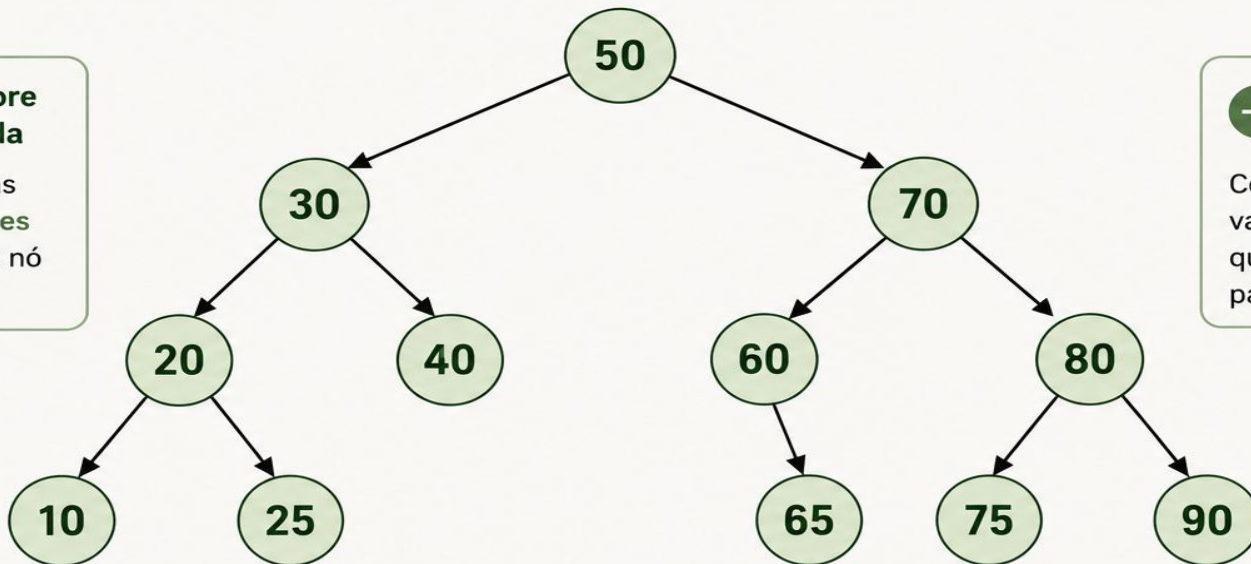
ÁRVORE BINÁRIA DE BUSCA

Em uma Árvore Binária de Busca (ABB), para cada nó:
todos os valores à esquerda são menores e todos os valores à direita são maiores.



Subárvore Esquerda

Contém apenas valores **menores** que o valor do nó pai.



Subárvore Direita

Contém apenas valores **maiores** que o valor do nó pai.



Busca na ABB

Para buscar um valor, comparamos com o nó atual:

- Se for menor, seguimos para a subárvore da esquerda.
- Se for maior, seguimos para a subárvore da direita.
- Se for igual, encontramos o valor.



Operações eficientes:
busca, inserção e remoção
em média $O(\log n)$.

ÁRVORE BINÁRIA

■ É um tipo especial de árvore

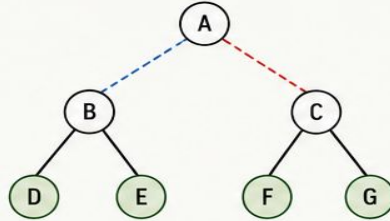
- Cada nó pode possuir nenhuma, uma ou no máximo duas sub-árvores
- Sub-árvore da **esquerda** e a da **direita**
- Usadas em situações onde, a cada passo, é preciso tomar uma decisão entre duas direções

Existem três tipos de árvores binárias

- Estritamente binária
- Quase Completa
- Cheia

- Nó interno
- Nó folha
- Sub-árvore da esquerda
- Sub-árvore da direita

DEFINIÇÃO



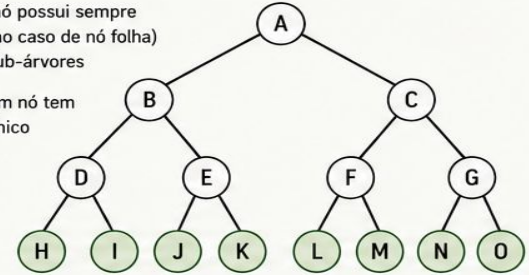
Cada nó pode ter:

- 0 filho (nó folha)
- 1 filho (apenas à esquerda ou à direita)
- 2 filhos (esquerda e direita)

A cada nó interno partem no máximo 2 ramificações:
sub-árvore da **esquerda** e sub-árvore da **direita**.

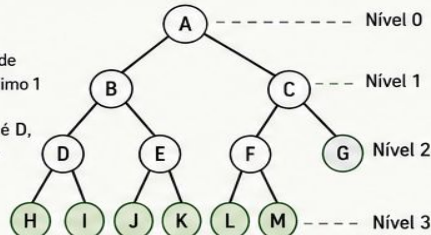
1. ÁRVORE ESTRITAMENTE BINÁRIA

- Cada nó possui sempre ou 0 (no caso de nó folha) ou 2 sub-árvores
- Nenhum nó tem filho único



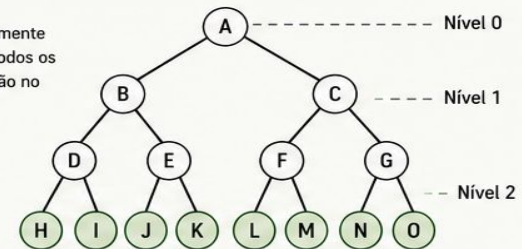
2. ÁRVORE BINÁRIA QUASE COMPLETA

- A diferença de altura entre as sub-árvores de qualquer nó é no máximo 1
- Se a altura da árvore é D, cada nó folha está no nível D ou D-1

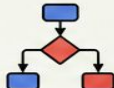


3. ÁRVORE BINÁRIA CHEIA (OU COMPLETA)

- Árvore estritamente binária onde todos os nós folhas estão no mesmo nível



APLICAÇÕES



Tomada de decisões
(ex.: jogos, algoritmos de decisão)



Sistemas de arquivos
e diretórios



Algoritmos de busca
e ordenação



Compiladores e
processamento
de expressões

Tipos de representação de árvore binária

TIPO DE REPRESENTAÇÃO

Como implementar uma árvore no computador?

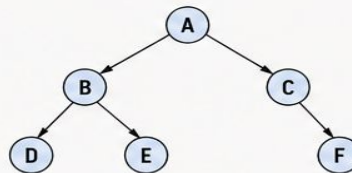
- Existem duas abordagens muito utilizadas
- Usando um array (alocação estática)
- Usando uma lista encadeada (alocação dinâmica)



1. USANDO UM ARRAY (ALOCAÇÃO ESTÁTICA)

- Necessário definir o número máximo de nós
 - Tamanho do array
- Usa 2 funções para retornar a posição dos filhos à esquerda e à direita de um pai

Exemplo de árvore



Representação em array

Índice:	0	1	2	3	4	5	6
Array:	A	B	C	D	E	F	-

Fórmulas (índice i do nó pai)

- Filho à esquerda: $2 \times i + 1$
- Filho à direita: $2 \times i + 2$
- Pai: $(i - 1) / 2$

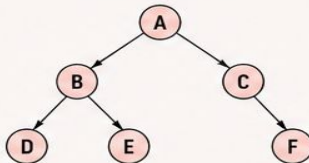
Legenda

- Nó ocupado
- Posição vazia

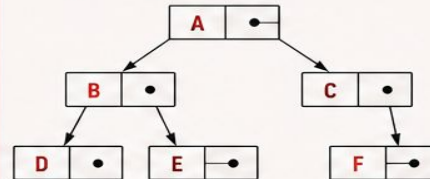
2. LISTA ENCADEADA (ALOCAÇÃO DINÂMICA)

- Espaço de memória alocado em tempo de execução
 - A árvore cresce à medida que novos elementos são armazenados, e diminui à medida que elementos são removidos

Exemplo de árvore



Representação em lista encadeada



Estrutura de um nó

esq	info	dir
-----	------	-----

esq: ponteiro para o filho à esquerda
info: informação do nó
dir: ponteiro para o filho à direita



COMPARAÇÃO ENTRE AS ABORDAGENS

Aspecto	Array (estática)	Lista encadeada (dinâmica)
Alocação de memória	Tamanho fixo (definido antecipadamente)	Tamanho variável (em tempo de execução)
Uso de memória	Pode desperdiçar espaço	Usa apenas o necessário
Inserção/remoção	Pode exigir realocação/cópia	Mais eficiente (apenas ajustes de ponteiros)
Implementação	Mais simples	Mais flexível, porém mais complexa

QUANDO USAR?

- **Array:** quando o número máximo de nós é conhecido e não muito grande.
- **Lista encadeada:** quando a árvore pode crescer/diminuir dinamicamente ou quando o número de nós é grande.